

MagicCode Super-Resolution Software FAQ and Troubleshooting

Document Version: v2.0.0. Release Date: September 2, 2026.

1. General

What does MagicCode Super-Resolution do?

It upscales images or GPU frames. v2.0.0 public modes are SPATIAL_SPEED_MODE, SPATIAL_BALANCED_MODE, TEMPORAL_SPEED_MODE, and TEMPORAL_BALANCED_MODE.

Does the software upload my images or videos?

No. Technical reporting does not upload image or video content. See the Privacy Policy and Data Reporting Notice (unchanged by this technical release).

What scale factors are supported?

Spatial: [1.0, 8.0]. Temporal: (1.0, 8.0]. Actual availability depends on backend and model. API support is not a claim that every scale is device-run.

2. Temporal FAQ

Can I run temporal through MC_Enable?

No complete temporal path. Enable has one color texture and no depth/motion/jitter. Use MC_Init / MC_Process with temporal_frame_t.

If I omit masks, do I have a semantic transparency mask?

No. The library derives an approximate mask when inputs are missing. Explicit textures win per channel. FSR-family paths bind a zero transparency texture when omitted and do not generate semantic transparency from color.a.

Which backends support temporal?

Android Vulkan/OpenGL ES, Apple Metal, Windows Vulkan / Direct3D 11 / OpenGL 4.3. CPU temporal is unavailable.

Temporal Super-Resolution

v2.0.0 adds temporal super-resolution through the session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + MC_Init / MC_Process / MC_Uninit. MC_Enable remains a single-texture spatial

helper. It may accept a temporal `alg_mode_e` value by range check, but it cannot supply per-frame depth, motion, or jitter and must not be documented as a complete temporal entry.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal `scaler_factor` is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- `TEMPORAL_SPEED_MODE` on Metal, Vulkan, and OpenGL currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- `TEMPORAL_BALANCED_MODE` uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGL do not run edge-resolve. If `enable_sharpening` is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- `magic_frame_t.image_in` / `image_out`: caller-owned color and output GPU resources.
- `magic_frame_t.frame`: non-NULL `temporal_frame_t*` with `struct_size = sizeof(temporal_frame_t)`.
- `temporal_frame_t.depth` and motion at input resolution.
- `jitter_offset_x/y` in input pixels, top-left, +X right, +Y down.
- `frame_index` / `reset_history`; `camera_near` / `camera_far` / `camera_fov_y` (radians).
- `gpu_context` at `MC_Init`: Vulkan requires `physical_device` + `device`; D3D11 requires `device`; Metal `device` may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires `magic_frame_t.command_buffer` to be a recording `VkCommandBuffer`. Metal `command_buffer` is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage `depth_reversed` / `depth_infinite` / `hdr_color` on `input_param_t` (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- `motion_vector_scale_x/y` is the only MV numeric conversion. (0,0) defaults to (`input_width`, `input_height`). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- `reactive` / `transparency` are optional. Explicit caller textures win per channel.

- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

3. Initialization problems

- Model not found: SetModelPath/SetModelDir, packaging, permissions.
- Width/height out of range: 64..4096.
- Scale out of range: spatial [1,8], temporal (1,8].
- Backend unavailable: CPU temporal, or GPU temporal not in this build.
- Vulkan/D3D11 GPU create failed: missing gpu_context.device (and Vulkan physical_device).

4. Processing problems

- MC_ERROR_TEMPORAL_NULL_FRAME: magic_frame_t.frame is NULL.
- Missing color/output/depth/motion handles.
- Vulkan command_buffer NULL or layout UNDEFINED.
- Ghosting: reset_history on hard cuts; check MV current→previous and +Y down jitter.
- Wrong output size: QUERY_STATUS and confirm scaler_factor.

5. Unity / Unreal

- DllNotFoundException: place the native plugin under Assets/Plugins/Android/arm64-v8a/.
- UE missing library: lib/android and lib/ios libmagic_sr_enable.a (not a legacy release/v1.1 path).
- Plugin Enable cannot feed temporal extras; use Native session API for temporal.

6. Diagnostic checklist

- MC_GetVersion should report v2.0.0.
- Record backend, input type, alg_mode, scale, width, height, model path.
- Record API return values and output_status_params_t.error_code.
- For temporal, confirm temporal_status_valid after a successful process.